📘 Playwright Execution Flow Notes

```
import { test } from "@playwright/test";

test("RBG demo", async ({ page }) => {

    let userId: string = "RBG technologies";

    console.log(`This is demo at ${userId} for new batch`);

    await page.goto("http://gmail.com");

    await page.fill("#identifierId", userId);

    await page.pause();

    console.log(`This is demo at ${userId} for new batch`);

});
```

Step-by-Step Execution

1. Import Playwright Test

```
import { test } from "@playwright/test";
```

Purpose

- Imports the `test()` function from Playwright.
- `test()` is used to create and execute test cases.

---

2. Create a Test Case

```
test("RBG demo", async ({ page }) => {
```

Purpose

- Creates a test named **"RBG demo"**.

- Playwright automatically executes this test.

What happens internally?

Playwright:

- Launches the browser.
- Creates a new Browser Context.
- Creates a new Page (Browser Tab).
- Passes the `page` object to your callback.

Internally (simplified):

```
const browser = launchBrowser();

const context = browser.newContext();

const page = context.newPage();

callback({ page });
```

> **Note:** You do **not** create the browser or page manually. Playwright creates them automatically through its fixtures.

---

3. Browser Launch

```
test("RBG demo", async ({ page }) => {
```

Which line launches the browser?

There is **no line in your code that explicitly launches the browser**.

When Playwright starts executing the test, it automatically:

- Launches the configured browser (Chromium, Firefox, or WebKit).
- Creates a browser context.
- Opens a new page.
- Passes the `page` fixture to your test.

---

4. Store the Username

```
let userId: string = "RBG technologies";
```

Purpose

- Declares a string variable.
- Stores the username value.
- This value will later be entered into the application.

---

5. Print Message

```
console.log(`This is demo at ${userId} for new batch`);
```

Purpose

- Prints a message in the terminal.
- Useful for debugging and understanding execution flow.

---

6. Open the Website

```
await page.goto("http://gmail.com");
```

Purpose

- Opens the specified URL.
- Waits until the page finishes loading.

What happens internally?

- Browser sends an HTTP request.
- Server returns the webpage.
- Playwright waits for the page to load before executing the next statement.

---

7. Enter Data

```
await page.fill("#identifierId", userId);
```

Purpose

- Finds the textbox using the CSS selector `#identifierId`.
- Clears any existing value.
- Enters the value stored in `userId`.

Equivalent to:

```
Click Textbox

↓

Clear Existing Text

↓

Type "RBG technologies"
```

---

8. Pause Execution

```
await page.pause();
```

Purpose

- Stops test execution.
- Opens Playwright Inspector.
- Allows you to inspect elements and debug the test.

---

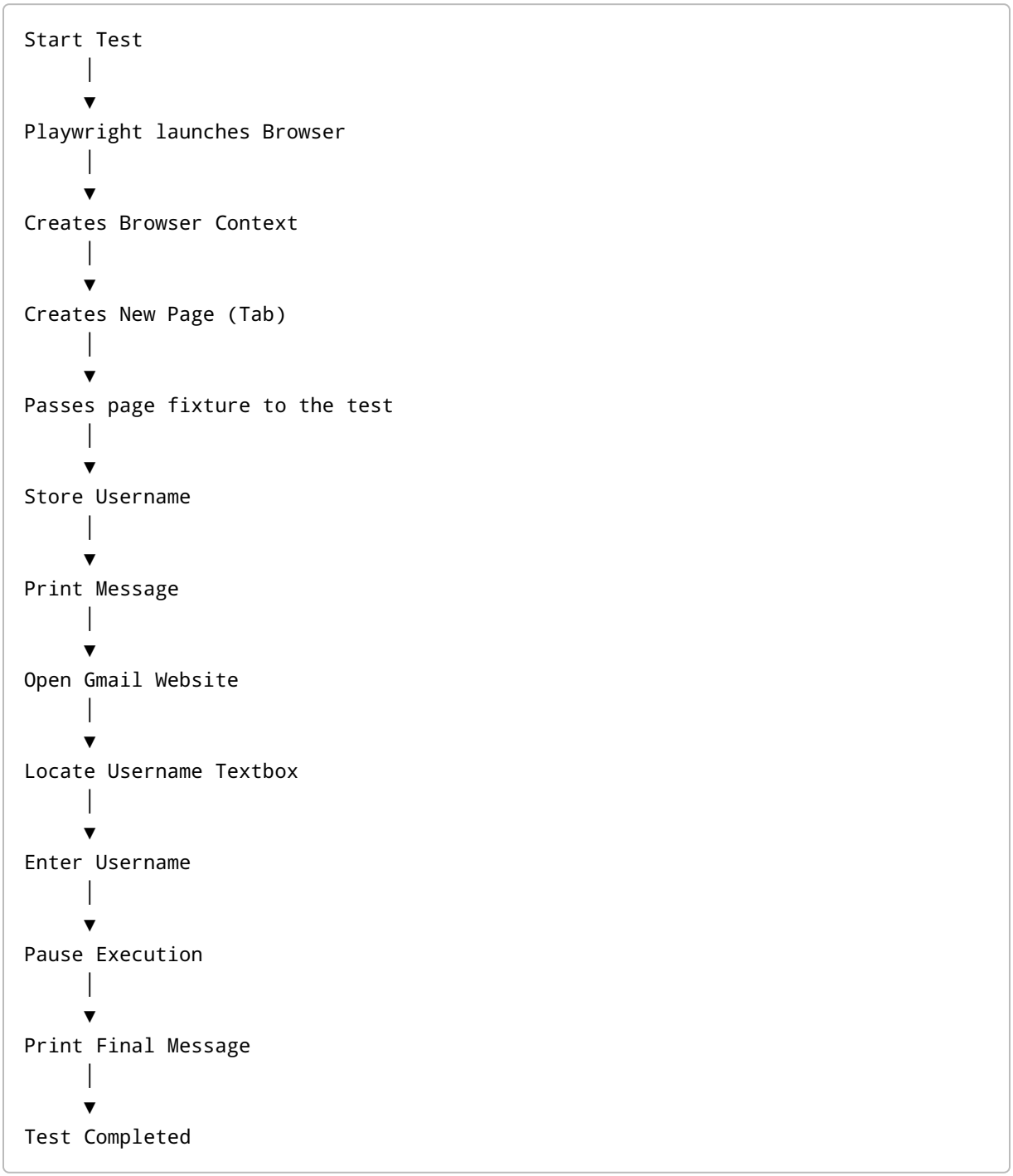
9. Print Final Message

```
console.log(`This is demo at ${userId} for new batch`);
```

Purpose

- Prints another message after the previous steps have completed.
- Useful for confirming that execution reached the end of the test.

Complete Execution Flow



Interview Questions

Q1. Which line launches the browser?

Answer:

There is no explicit line in the test. Playwright automatically launches the browser before executing the callback passed to `test()`.

Q2. Which line opens the website?

```
await page.goto("http://gmail.com");
```

Q3. Which line enters data into the textbox?

```
await page.fill("#identifierId", userId);
```

Q4. Where does the `page` object come from?

Answer:

The `page` object is automatically created and provided by Playwright through its built-in **Page fixture**.

Q5. What is the purpose of `page.fill()`?

Answer:

`page.fill()` locates an input element, clears any existing text, and enters the specified value into the textbox.